

1. What is the difference between Closed Source, Open Source, and Open Weight Models?

Large Language Models (LLMs) are released in different ways depending on how much access the creators provide. The three most common categories are **Closed Source**, **Open Source**, and **Open Weight** models.

Closed Source Models

Closed source models are completely controlled by the company that builds them. Users can access these models only through an API or the company's own application. The model's architecture, weights, training data, and training process are not publicly available.

Examples include **OpenAI's GPT-4**, **Anthropic's Claude**, and **Google Gemini**.

Advantages

- High performance
- Professionally maintained
- Regular updates
- Strong safety mechanisms

Disadvantages

- Cannot run locally
 - Requires API usage
 - Usage costs money
 - No visibility into how the model was trained
-

Open Source Models

A truly open-source model provides everything required to reproduce and improve it, including the model architecture, training code, model weights, and often the datasets used for training.

Examples include **OLMo (Allen AI)** and **Pythia (EleutherAI)**.

Advantages

- Complete transparency
- Anyone can inspect or modify the model
- Ideal for academic research

- Encourages community collaboration

Disadvantages

- Very few modern frontier models are fully open source
 - Training from scratch still requires enormous computational resources
-

Open Weight Models

Open-weight models occupy the middle ground. The trained model weights are publicly available, allowing users to download and run the model locally. However, the original training data, training pipeline, and exact methodology are generally not released.

Examples include **Meta Llama 3**, **Mistral 7B**, and **Google Gemma**.

These models are excellent for fine-tuning, research, and local deployment.

Advantages

- Can be run on personal hardware
- Can be fine-tuned for custom tasks
- No API dependency
- Large ecosystem support

Disadvantages

- Training process remains private
- Licences may restrict commercial usage
- Cannot fully reproduce the original model

Conclusion

Although many people refer to models like Llama as "open source," they are more accurately described as **open-weight models**. They provide access to the trained parameters but not the complete training recipe. For most students and developers, open-weight models are the best option because they can be downloaded, fine-tuned, and deployed locally without the cost of proprietary APIs.

2. One-Page Summary: Top 3 Tools for Model Training and Fine-Tuning

Training modern language models from scratch is beyond the reach of most individuals, but fine-tuning existing open-weight models has become much more accessible. Several frameworks simplify this process, with three standing out in today's ecosystem.

1. Hugging Face TRL (Transformer Reinforcement Learning)

TRL is built on top of the Hugging Face Transformers library and provides everything needed for supervised fine-tuning and alignment methods such as Direct Preference Optimisation (DPO) and Proximal Policy Optimisation (PPO).

It integrates seamlessly with PEFT libraries such as LoRA and QLoRA, making it an excellent choice for research and experimentation.

Best suited for:

- Learning fine-tuning
 - Instruction tuning
 - RLHF and preference optimisation
 - Research projects
-

2. Axolotl

Axolotl is a configuration-driven framework designed for reproducible fine-tuning. Instead of writing extensive Python training scripts, users define almost everything inside a YAML configuration file.

It supports a wide variety of models, quantisation methods, and distributed training techniques, making it popular among both researchers and production teams.

Best suited for:

- Production fine-tuning
 - Large custom datasets
 - Reproducible experiments
 - Advanced configurations
-

3. Unsloth

Unsloth focuses on making fine-tuning faster and more memory efficient. It uses highly optimised Triton kernels to reduce GPU memory consumption while significantly increasing training speed.

Even users with a single consumer GPU or Google Colab can fine-tune modern models effectively.

Best suited for:

- Students

- Local GPU users
 - Google Colab
 - Low-VRAM hardware
 - Rapid experimentation
-

Recommendation

If I were recommending a starting point for someone learning modern LLM development, I would suggest:

- **Unsloth** for beginners and students because it is fast, memory efficient, and simple to use.
- **TRL** when learning alignment techniques such as DPO or RLHF.
- **Axolotl** when building reproducible or production-oriented fine-tuning pipelines.

A practical learning path would be to begin with Unsloth, explore TRL to understand advanced training methods, and then move to Axolotl for more complex projects.

3. Why Can't We Train Large Language Models from Scratch?

A common misconception is that anyone can build a model similar to GPT-4 or Llama from scratch. In reality, pre-training a modern LLM requires enormous computational resources.

Training a frontier model involves processing trillions of tokens across thousands of high-end GPUs such as NVIDIA H100 or B200 accelerators. The process can take months and costs millions of pounds in hardware, electricity, storage, and engineering effort. It also requires carefully curated datasets collected from books, websites, scientific papers, and other sources.

For this reason, training a new LLM from scratch is generally beyond the reach of individual developers, students, and most organisations.

However, this does not prevent people from building powerful AI systems.

Instead of pre-training an entire model, developers use **Parameter-Efficient Fine-Tuning (PEFT)** techniques such as **LoRA** and **QLoRA**. These methods freeze the original model weights and train only a small number of additional parameters, greatly reducing computational requirements.

Combined with quantisation techniques that load models in 4-bit or 8-bit precision, developers can fine-tune models such as Llama 3, Gemma, or Mistral on a single consumer GPU or even free cloud platforms.

Today, most practical AI development is based on fine-tuning existing open-weight models rather than training entirely new ones.

4. One-Page Summary: Difference Between LLMs and SLMs

Large Language Models (LLMs) and Small Language Models (SLMs) share the same fundamental Transformer architecture, but they differ significantly in scale, cost, and intended use.

LLMs generally contain tens or even hundreds of billions of parameters. Their large size allows them to perform a wide range of tasks, including reasoning, coding, translation, creative writing, and question answering. Because of their size, they require multiple GPUs, large amounts of memory, and significant computational resources for both training and deployment.

SLMs, by contrast, typically contain between one and eight billion parameters. Rather than relying on sheer scale, they achieve strong performance through high-quality training data, synthetic data generation, and efficient optimisation techniques. They are faster, cheaper to deploy, and can often run on consumer hardware or edge devices.

The choice between an LLM and an SLM depends on the application. If the goal is broad general intelligence and complex reasoning across many domains, an LLM is usually preferable. If the goal is low latency, low cost, or deployment on laptops, mobile devices, or embedded systems, an SLM is often the better choice.

In recent years, improvements in training methods have allowed modern SLMs, such as Microsoft's Phi family and Mistral 7B, to achieve performance that rivals much larger models on many specialised tasks. This demonstrates that high-quality data and efficient training can sometimes compensate for smaller model size.

5. What Other Model Formats Are Available? What is GGUF?

After a model has been trained, it must be stored in a format that can later be loaded for inference. Several model formats are commonly used in the AI ecosystem, each designed for different deployment scenarios.

The most widely used format is **Safetensors**, which has become the standard within the Hugging Face ecosystem. Unlike older PyTorch checkpoint formats, Safetensors is designed to be secure because it cannot execute arbitrary code when loaded. It also supports faster loading and efficient memory mapping.

For GPU-based inference, formats such as **GPTQ** and **AWQ** are commonly used. These formats apply quantisation techniques that reduce model size while maintaining good accuracy, enabling large models to run on GPUs with limited memory.

Another important format is **ONNX**, which provides a framework-independent representation of machine learning models. ONNX allows models to be deployed across different hardware platforms and inference engines, making it popular in production environments.

One of the most popular formats for local inference is **GGUF (GPT-Generated Unified Format)**. GGUF was introduced by the `llama.cpp` project as the successor to the older GGML format.

GGUF packages the model weights, tokenizer, vocabulary, metadata, and architecture information into a single file. This makes deployment much simpler because everything required to run the model is stored together.

The format is designed to support efficient memory mapping, allowing models to load quickly without copying the entire file into memory. It also supports CPU inference and hybrid CPU-GPU execution, making it particularly suitable for laptops and consumer hardware.

As a result, GGUF has become the preferred format for running open-weight models locally through tools such as **llama.cpp**, **Ollama**, and **LM Studio**. For developers who want to run models on their own machines without relying on cloud APIs, GGUF offers an efficient and practical solution.